



HTML Intro



Intro

Q: How do we use the browser to navigate the internet?

Where does the content you see when you open a webpage come from?

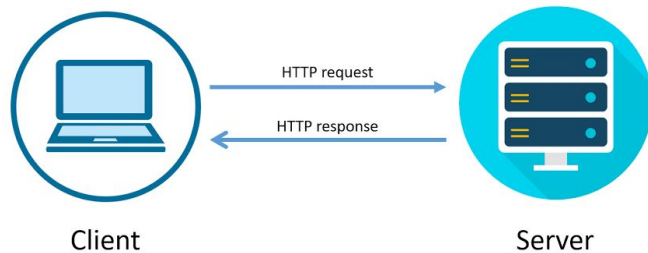
How does a website load and display in your browser?



Tell me what you want...

The internet is nothing more than a bunch of computers constantly connected, waiting for you to ask them for something

These computers, also called **servers**, will respond to your browser **requests** with the necessary information to display a page in your browser

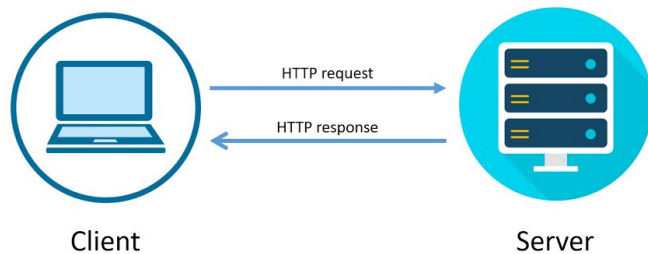




Tell me what you want...

The browser, also known as the **client**, will then display the **response**, and sit patiently waiting for the user's next **input**

This is known as the **request/response cycle**, and it's how you interact with every single page in the internet!





Time for some live coding

Let's look in closer at how the browser makes a request!

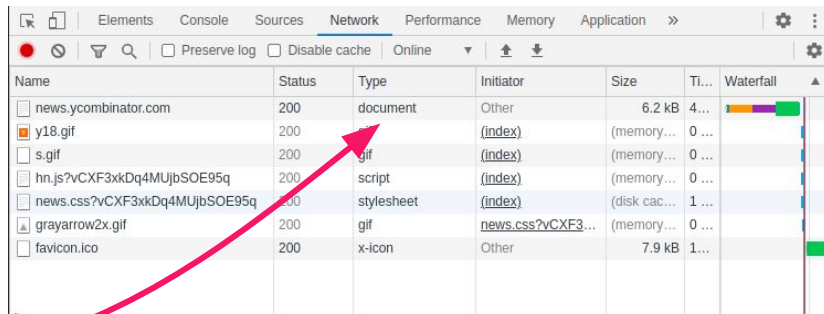
Check out [Hacker News](#) network tab in the console



Let's peek at the response

As we discussed before, all the browser does is ask a server for the file that contains the page you want to visit

We can think of this as going into a specific **folder** on our laptop, and **opening** a file that we want to see!



Name	Status	Type	Initiator	Size	Ti...	Waterfall
news.ycombinator.com	200	document	Other	6.2 kB	4...	
y18.gif	200	gif	(index)	(memory...	0 ...	
s.gif	200	gif	(index)	(memory...	0 ...	
hn.js?vCxF3xkDq4MUjbSOE95q	200	script	(index)	(memory...	0 ...	
news.css?vCxF3xkDq4MUjbSOE95q	200	stylesheet	(index)	(disk cac...	1 ...	
grayarrow2x.gif	200	gif	news.css?vCxF3...	(memory...	0 ...	
favicon.ico	200	x-icon	Other	7.9 kB	1...	



Let's peek at the response

After the browser loads and **parses** the file, it will look up what other **resources** it needs to display the page properly

It will look at the **tags** declared in this first file, and **fetch** all the content missing.

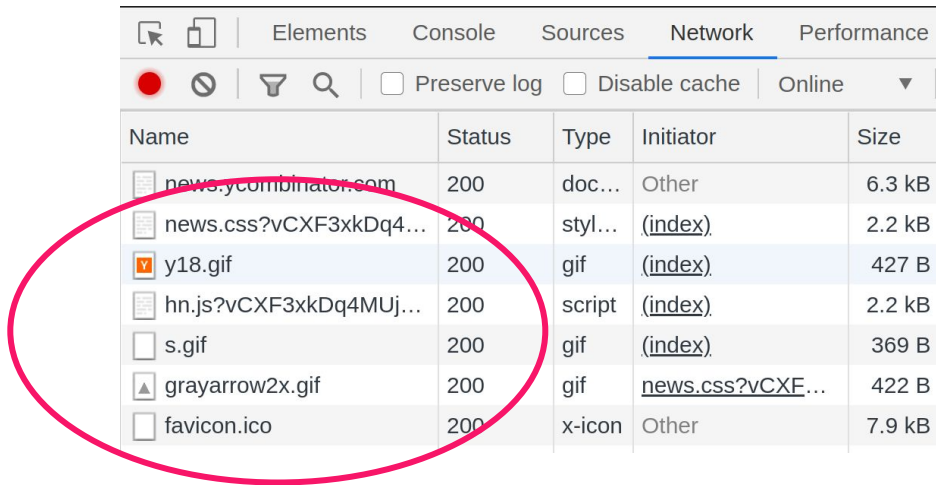
```
<html lang="en" op="news">
  <head>
    <meta name="referrer" content="origin">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" type="text/css" href="news.css?vCXF3xkDq4MUjbS0E95q">
    <link rel="shortcut icon" href="favicon.ico">
    <link rel="alternate" type="application/rss+xml" title="RSS" href="rss">
    <title>Hacker News</title>
  </head>
  <body == $0
    <center>
      <table id="hnmain" border="0" cellpadding="0" cellspacing="0" width="85%" bg
    </center>
    <script type="text/javascript" src="hn.js?vCXF3xkDq4MUjbS0E95q"></script>
  </body>
</html>
```



Let's peek at the response

A bunch of other requests are made by the browser, then the page is ready to **render**!

Going back to our folder analogy, it's as if you've just opened a word document to show it to your friends, that includes those precious images that you've spent hours looking for and you're so proud of!

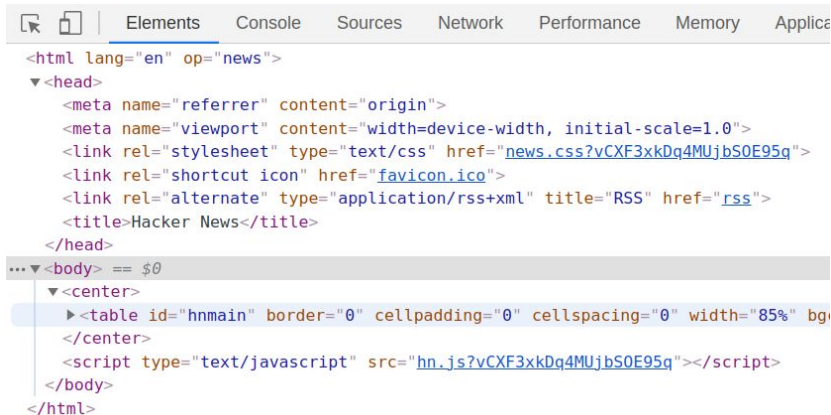


Name	Status	Type	Initiator	Size
news.ycombinator.com	200	doc...	Other	6.3 kB
news.css?vCXF3xkDq4...	200	styl...	(index)	2.2 kB
y18.gif	200	gif	(index)	427 B
hn.js?vCXF3xkDq4MUj...	200	script	(index)	2.2 kB
s.gif	200	gif	(index)	369 B
grayarrow2x.gif	200	gif	news.css?vCXF...	422 B
favicon.ico	200	x-icon	Other	7.9 kB



Tag?

Q: We mentioned tags earlier
Could you explain what they are?
Why do you think we need them?



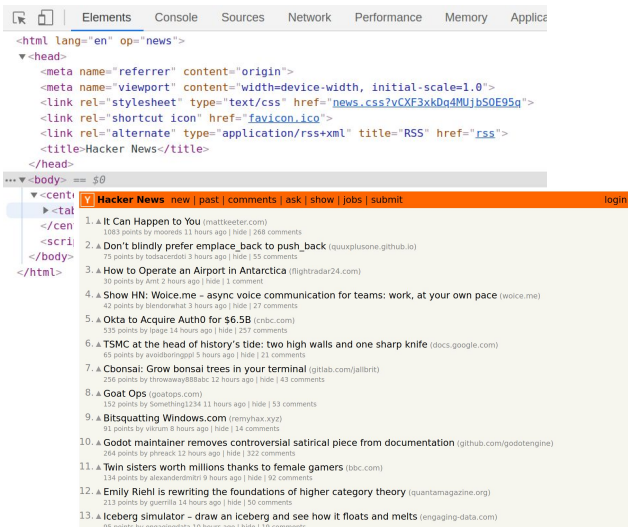
```
<html lang="en" op="news">
  <head>
    <meta name="referrer" content="origin">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" type="text/css" href="news.css?vCXf3xkDq4MUjbS0E95q">
    <link rel="shortcut icon" href="favicon.ico">
    <link rel="alternate" type="application/rss+xml" title="RSS" href="rss">
    <title>Hacker News</title>
  </head>
  <body> == $0
    <center>
      <table id="hnmain" border="0" cellpadding="0" cellspacing="0" width="85%" bg
    </center>
    <script type="text/javascript" src="hn.js?vCXf3xkDq4MUjbS0E95q"></script>
  </body>
</html>
```



A: These are part of a Markup language called **HTML** (Hyper **T**ext **M**arkup **L**anguage)

Instead, the purpose of a markup language is to have a **uniform notation** that describes **content** and **structure**

10





HTML Markup

```
<tag>I'm a tag!</tag>
```

The basic building block of HTML is the **tag**. We've seen a few already in the examples before, but here we have them in its simplest form.



HTML Markup

```
<tag>I'm a tag!</tag>
```

Here we have the **opening tag**, which is just the the tag name wrapped in angled brackets



HTML Markup

```
<tag>I'm a tag!</tag>
```

Here we have the **closing tag**. The main difference is the **forward slash**, to tell us that this is the closing tag



HTML Markup

```
<tag>I'm a tag!</tag>
```

Then in between the tags we've got the **content** belonging to this tag

Remember that the purpose of tags is to describe the structure and form of the content

This means that its content doesn't have to be only text!



HTML Markup

```
<tag>I'm a tag!</tag>
```

```

```

```
<br/>
```

Tags are meant to be used in **pairs**. But there are some specific ones that don't have a closing counterpart

We can distinguish them by the forward slash before the last angled bracket, and they're called **self-closing tags**

Q: What's the difference between these three tags?



HTML Markup

```
<tag>I'm a tag!</tag>
```

```

```

```
<br/>
```

A: These are called **attributes**. They will allow you to declare extra **properties** that belong to the tag



HTML Markup

```
<p contenteditable>Hello!</p>
```

```

```

Attributes will either have a **name** followed by an **=** sign and a **"string"** with its value

Or they will only have their **name** on its own that expresses that this attribute is **enabled**



HTML Markup

```
<tag>I'm a tag!</tag>
```

```

```

```
<br/>
```

The combination of the open and closing **tags**, with its **attributes**, plus any **content**, forms what's called an **element**



HTML Markup

```
<div>  
  <div>  
    <div></div>  
  </div>  
</div>
```

You can have elements inside of other elements as well!

This is called **nesting**

This will allow you to group elements together in a meaningful way!



HTML Elements

```
<h1></h1>
```

```
...
```

```
<h6></h6>
```

```
<p></p>
```

```
<div></div>
```

Now that we've seen what HTML markup looks like, let's have a look at some of the most **basic elements**

Since originally HTML was created to model a real world written document, you'll notice that some of these elements are familiar in name, and how they behave when your HTML page **renders**



HTML Elements

```
<h1>Title!</h1>
```

```
<h2>Title!</h2>
```

```
<h3>Title!</h3>
```

```
<h4>Title!</h4>
```

```
<h5>Title!</h5>
```

```
<h6>Title!</h6>
```

These are **heading** elements, and they create a hierarchy for the titles in a page





HTML Elements

```
<h1>We have paragraphs!</h1>
```

```
<p>
```

```
  Lorem ipsum dolor sit, amet  
  consectetur adipisicing elit. Animi  
  voluptatibus error, accusamus, nihil  
  itaque possimus ipsum consequuntur  
  quam blanditiis incidunt fugiat  
  similique molestias rem nobis.  
  Molestiae ratione reiciendis nostrum  
  facilis?
```

```
</p>
```

Next we have the **paragraph** element. It does what it says on the tin... divides chunks of text into paragraphs





HTML Elements

```
<div>
```

```
<h1>We have paragraphs!</h1>
```

```
<p>
```

```
  Lorem ipsum dolor sit, amet
```

```
  consectetur adipisicing elit. Animi
```

```
  voluptatibus error, accusamus, nihil
```

```
  itaque possimus ipsum consequuntur
```

```
  quam blanditiis incidunt fugiat
```

```
  similique molestias rem nobis.
```

```
  Molestiae ratione reiciendis nostrum
```

```
  facilis?
```

```
</p>
```

```
</div>
```

This is a **div** element. They're just an empty **box** that we use to group elements together, for various purposes

Q: Can you see any differences between this and the previous example?





HTML Elements

```
<div>
```

```
<h1>We have paragraphs!</h1>
```

```
<p>
```

```
  Lorem ipsum dolor sit, amet
```

```
  consectetur adipisicing elit. Animi
```

```
  voluptatibus error, accusamus, nihil
```

```
  itaque possimus ipsum consequuntur
```

```
  quam blanditiis incidunt fugiat
```

```
  similique molestias rem nobis.
```

```
  Molestiae ratione reiciendis nostrum
```

```
  facilis?
```

```
</p>
```

```
</div>
```

A: There's no difference between them! Divs don't have **any visible features** by default

However, these **container elements** will become a crucial part of your web app building tool, you will see why soon!





Time for some practice

It's time to set up our first HTML page!

Play around with the elements covered so far

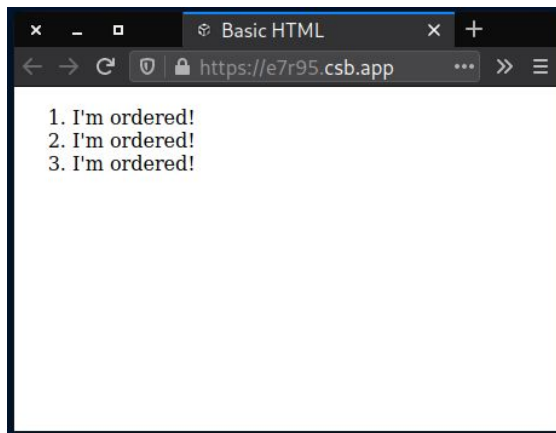


HTML Elements

There's also some elements that allow you to structure your text in different ways

One of them is the **ordered list**, notice how we nest each element with a **list item**

```
<ol>
  <li>I'm ordered!</li>
  <li>I'm ordered!</li>
  <li>I'm ordered!</li>
</ol>
```





HTML Elements

```
<p>
```

```
Here I am, some text waiting to be
```

```
<strong>bold</strong> and <em>italic</em>.
```

```
<em><strong>Sometimes I feel like being both!
```

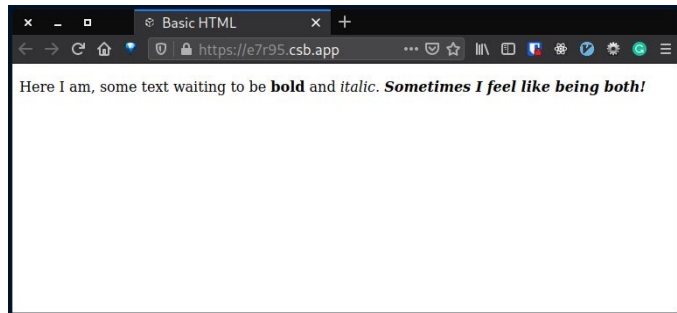
```
</strong></em>
```

```
</p>
```

Here we have the **strong** and **em** elements, that will make the text they contain **bold** or *italic*

By **nesting** them, we have both effects applied to the same text!

Q: Does the paragraph text looks the same between the snippet and the page?





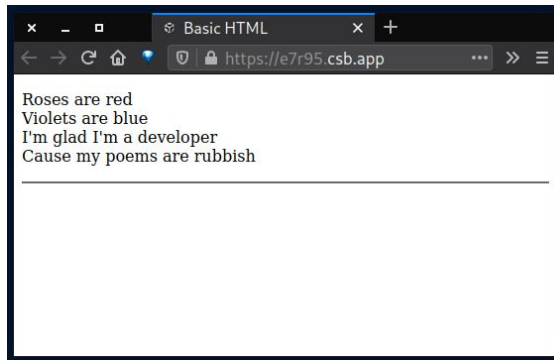
HTML Elements

```
<p>  
Roses are red <br/>  
Violets are blue <br/>  
I'm glad I'm a developer <br/>  
Cause my poems are rubbish  
</p>  
<hr/>
```

A: Not really.

If we want to **break** a block of text at a specific point, we need to also use HTML. For that we have the **br** element.

Other times you might want to create a separation in between sections of your page, so we have a **horizontal ruler**





HTML Elements

```
<p>
```

```
This is the element that allows you to click in  
certain parts of a page, and go to a different  
page, like
```

```
<a
```

```
href="https://developer.mozilla.org/en-US/docs/Web/  
HTML/Element/a">here</a>!
```

```
</p>
```

To be able to navigate to other pages, we use the **anchor** element

This is the element that allows you to click in certain parts of a page, and go to a different page, like [here!](#)

The **href** attribute contains the address you want users to navigate to



HTML Elements

```
<p>  
  This is the element that will load images to the  
page <br />  
  <br />  
  
</p>
```

We can load images into our pages with **img**

We need to set the **src** (source) attribute with an **url** to tell the browser which image to load.

For now we're just going to use full urls, but we'll see later we can load images from different kind of sources.



HTML Elements

```
<p>  
  This is the element that will load images to the  
page <br />  
  <br />  
  
</p>
```

We should as well set the **alt** attribute with a description of the image

Optionally, we can also resize the image with the **width** and **height** attributes, in pixels



Find yourself stuck?



Google is your friend!

All the questions have been asked before by someone out there, so its most likely you'll find answers if you google the **right keywords**

Also, try always to **refer back to the official documentation** of any technology you're using.

For HTML, CSS and JavaScript, it will be the **MDN** website

To be more efficient, you can also combine the name of the docs website with the term you're searching for, like:

MDN p or **MDN anchor tag**



Basic Setup

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport">
    <title>Document</title>
  </head>
  <body>

  </body>
</html>
```

Now that we know how to write HTML, we just need to know where to put it

By **convention**, the file that is the entry point for our page, is called **index.html**

This will be the file that a browser will load when someone looks for our page online



Basic Setup

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport">
    <title>Document</title>
  </head>
  <body>

  </body>
</html>
```

Now that we have our index.html file, we need to start writing our HTML in a place that the browser understands

This place is inside the **body** element!

Everything we saw so far will go in **between** these two tags. You might have noticed a few other things outside the body element, and we'll go back to them later.

Q: What do you think will happen if we put anything else other than HTML in the body?

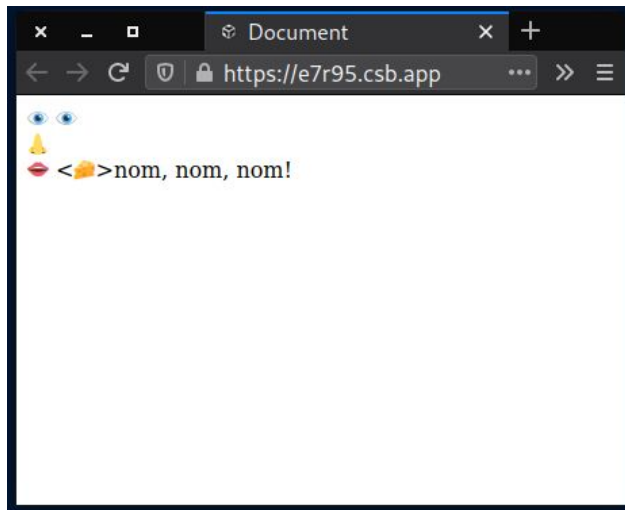


Basic Setup

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" >
    <title>Document</title>
  </head>
  <body>
    <eye>👁️</eye>
    <eye>👁️</eye><br />
    <nose>👃</nose><br />
    <mouth>👄</mouth>
    <👉>nom, nom, nom!</👉>
  </body>
</html>
```

A: Anything that you put there, the browser will try it's best to put it on the page. If it can't understand it, it will just ignore it

Which means it will **not warn** you if you made a mistake!





Quick pitstop; My code is out of control!

Now that we wrote our first serious piece of code, we start to realise how quickly it can become **messy, hard to read**, and **hard to understand**!

So this is a good time to introduce some good practices for when you're out and about writing your own code



Quick pitstop; My code is out of control!

One common practice to help you manage this is to **write comments in your code**

You can use comments to:

- Separate sections of the codebase
- Describe implementation details
- Leave warnings about potential pitfalls on the code

Comments are especially useful when you work in a collaborative environment, or for the future you

Word of caution! Don't over rely on comments to help you organise or document your codebase. They can quickly become a nuisance, and just add to the problem we're trying to solve

```
<body>
  <!-- this is what a comment looks like -->

  <!-- You can have them
        even over
        multiple lines -->
</body>
```



Quick pitstop; My code is out of control!

With **indentation** we aim to create a visual hierarchy that will make it easy to read and understand our codebase, even at a glance!

This is a detail usually decided at the beginning of a project to ensure **consistency**

Q: Could you describe what is this snippet doing?

```
<div><ul><li>I'm an item</li><li>
I'm an item</li><li><a href="">
I'm a url</a></li></ul>
</div>
```



Quick pitstop; My code is out of control!

Q: How about now?

Here we see how proper **indentation** and nested elements can make our code so much **easier** to read and understand!

```
<div>
  <ul>
    <li>I'm an item</li>
    <li>I'm an item</li>
    <li>
      <a href="">I'm a url</a>
    </li>
  </ul>
</div>
```



Quick pitstop; My code is out of control!

Use **spacing** between different sections of your code to improve readability

As we've seen before, spaces in our code doesn't affect the way it runs and display, so we should use it to give some **breathing room** to our document.

At the end of the day, writing and reading code is just that, writing and reading!!

```
<div>

  <ul>
    <li>I'm an item</li>
    <li>I'm an item</li>
    <li>
      <a href="">I'm a url</a>
    </li>
  </ul>

</div>
```



Time for some practice

Let's create a profile page!

It should have the following elements:

- two different types of headings
- A couple of paragraphs
- Two lists (they can be ordered or unordered)
- Two images
- Use divs to create at least 3 different sections



A quick detour about metadata

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible">
    <meta name="viewport">
    <title>Document</title>
  </head>
  <body>

  </body>
</html>
```

The document starts at the **html** element. This is also called the **root** element, and all the other elements are its **children**

You might be asking yourself, what about the **DOCTYPE** tag?

That's not really an HTML tag, and it's just there to help our browser understand this is an HTML file

If you are curious, you can read more about it [here](#)



A quick detour about metadata

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible">
    <meta name="viewport">
    <title>Document</title>
  </head>
  <body>

  </body>
</html>
```

Then we have the **head** element. Be careful not to mistake it with any other header or heading elements we've seen before!

This element will never display elements on the page. Its purpose is to give more information to the browser about our page, also called **metadata**!

This element will also contain the **information** the browser needs to know to **load** the page properly



A quick detour about metadata

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible">
    <meta name="viewport">
    <title>Document</title>
  </head>
  <body>

</body>
</html>
```

Most of this information will be declared in the **meta** element

You can see that by changing the **attributes** of meta, you can declare different settings and give different pieces of information to the browser

Here we have some of the most common, like **charset**, the initial scale of the page/**viewport**, and a **compatibility settings** for Internet Explorer (yes, it is not yet extinct!).

Check out the docs about meta [here](#) to find out more about it



A quick detour about metadata

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible">
    <meta name="viewport">
    <link rel="stylesheet" href="style.css">
    <title>Document</title>
  </head>
  <body>

</body>
</html>
```

There's also the **title** element, where we set the title of the page that will appear in the browser's tab

Here we have another common element in the head, **link**

This is not the kind of link you click on a page, but rather a link that tells the browser it needs to go **fetch** another file to properly render the page

Q: Do you think you could guess what kind of file we're loading here, just by looking at the attributes of the link element?



CSS Intro

So far we've seen how we write HTML and structure our text in the page

By now we already know:

- **How to create an HTML page**
- **How to render elements on a page**
- **How to organise elements to give some structure to a page**

But we're still not able to change its look, to style it. After all, the web pages we use daily look nothing like the ones we've created so far...

We need **CSS** (Cascading **S**tyl**S**heets) for that!



CSS Intro

First, we need to figure out where we should write it

There are 3 ways you can do this:

- **Inline**
- Inside the document's **head** tag
- In a CSS **file**

There are use cases for all of this ways, but the most accepted as good practice is to write your CSS file

This way will be easier for us to keep **separation of responsibilities** in our code, **reuse files** if we need to, and make it **easier for us to debug**

Q: How can we add a CSS file to our HTML page? 🤔



CSS Intro

A: You're probably thinking "We've seen that already!", and you're right!

We use the **link** tag with the **rel** and **href** attributes, which declares we're loading a stylesheet and where it's located, respectively

Have a look [here](#) to learn more about it!

Also, you can read about how to write your CSS in the head element by using the **style** tag [here](#), and how to use **inline** CSS with the style property [here](#)

It is considered best practice to have an external stylesheet, so this is the method we're going to focus on

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible">
    <meta name="viewport" ">
    <link rel="stylesheet" href="style.css">
    <title>Document</title>
  </head>
  <body>

</body>
</html>
```



CSS Intro

We can think of **HTML** as the **skeleton** of our page, holding everything in place, but looking really bare!

Now we need to cover it with **skin** and make it **look good**, by using **CSS**

For that, we first need to target **individual parts** of the skeleton

Q: Looking at this HTML, can you come up with a way to say with code that you want to change the looks of the **paragraph** and the **list**?

```
<div>
  <h1>Here to style some elements!!</h1>
  <p>
    Lorem ipsum dolor sit amet.
  </p>
  <ol>
    <li>One</li>
    <li>Two</li>
    <li>three</li>
  </ol>
</div>
</body>
```



CSS Intro

A: You most likely came up with your version of **CSS Selectors!**

You can say which element you want to change by referencing the **tag** name, and then you just declare what you want to change with **CSS rules**

These come in pairs, first the **rule name** you're applying, then the **value**, separated by a **colon**

We end each rule with a **semicolon**

```
p {  
  color: red;  
}
```



Time for some live coding

Let's apply some CSS to our book!



Time for some practice

Apply some CSS to the profile page you created in the previous exercise!

You should:

- Change the color of the headings
- Change the color of the background
- Change the size of the font
- **Extension!** Remove the lists' items markers
- **Extension!** Use the MDN resource to find a different CSS property to the ones mentioned here and use it in your page